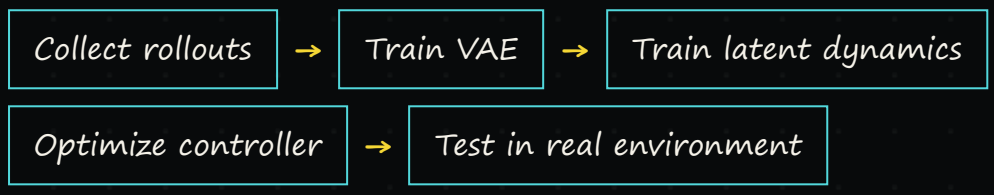
World Models

Compress observations, model latent dynamics, and learn a controller. A learned environment can make policy search cheaper, but policies must still work outside the generated dream.

Original study notes based on David Foster, Second edition, 2023. Book pp. 331–358. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.



Detailed Notes

Reinforcement learning and CarRacing

Source: book pp. 331–335 / PDF p. 359

In reinforcement learning an agent observes a state, chooses an action, receives rewards, and tries to maximize cumulative return. Observations may only partially reveal the underlying state, so history can matter.

CarRacing provides visual observations and continuous driving actions, making raw image-to-action learning challenging. A world model separates perception, dynamics, and control. The project's simulator is the real evaluation environment relative to the learned dream model; "real" here does not mean deploying a physical vehicle.

Architecture and staged training

Source: book pp. 336–339 / PDF p. 364

The system has a vision model V, memory/dynamics model M, and controller C. A VAE compresses images into z; a recurrent mixture-density model predicts future z from history and actions; a small controller selects actions from latent state and recurrent memory.

Training proceeds in stages: collect environment rollouts, train visual compression, encode transitions, train dynamics, then optimize controller parameters. Keeping perception/dynamics fixed during controller search makes the policy problem much smaller. Rollout coverage limits what the learned environment can represent.

Training and exploring the VAE

Source: book pp. 340–345 / PDF p. 368

Train the VAE on observations collected from rollouts, preserving the normalization and architecture used later. Test reconstructions and latent variation to see which road features and hazards survive compression.

A compact code that reconstructs average appearance may still omit details needed for control. Evaluate the representation by task-relevant information, not only visual aesthetics. Encode held-out trajectories and check whether unseen track situations are represented sensibly.

MDN-RNN data, architecture, and sampling

Source: book pp. 346–348 / PDF p. 374

Prepare sequences of latent codes, actions, and next-code targets. The recurrent hidden state summarizes history; a mixture-density head predicts mixing weights, means, and scales for possible next latent states.

> $p(z_{t+1} \mid z_t, a_t, h_t) = \Sigma_k \, \pi_k N(\mu_k, \Sigma_k)$

Train with mixture negative log likelihood, using stable log-sum-exp and positive scales. Multimodality matters when several futures are possible. During sampling, select a mixture component and draw a latent state, updating recurrent memory consistently.

Controller and CMA-ES

Source: book pp. 348–352 / PDF p. 376

The controller is intentionally small, commonly mapping latent code plus recurrent memory to continuous actions. This makes black-box optimization over its parameters practical.

Covariance Matrix Adaptation Evolution Strategy (CMA-ES) samples candidate parameter vectors, evaluates episode returns, and updates a search distribution toward better candidates. It does not backpropagate through the environment. Multiple rollouts/seeds reduce noise, and independent candidate evaluations can run in parallel. Compare average and worst-case behavior rather than choosing one lucky episode.

In-dream training and simulator exploitation

Source: book pp. 353–358 / PDF p. 381

Use the learned dynamics as the environment for controller search, reducing dependence on expensive environment execution. Appropriate stochasticity/temperature makes the dream less deterministic and may reduce exploitation of modeling errors.

A controller can discover unrealistically rewarding actions that exploit a flawed world model. Strong dream performance is therefore not sufficient: evaluate the same policy in held-out actual-environment episodes and inspect differences. Distribution shift, compounding dynamics error, omitted hazards, and inaccurate termination/reward models limit transfer. Historical results illustrate feasibility, not safety certification.

Revision Check

1. What does each V/M/C component do?
2. Why use a mixture density instead of one predicted latent point?
3. How does CMA-ES differ from gradient descent?
4. How can a controller exploit a learned environment?
5. What must be validated before any physical deployment?

A learned world is a useful training instrument, not ground truth. Transfer validation and coverage of important failures remain essential.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.